

LLM FINE-TUNING

Complete Notes (Roman Urdu)

Pretraining se lekar Instruction Tuning, Preference Alignment (RLHF/DPO), LoRA/QLoRA, Hugging Face, Unsloth, Multimodal aur GPT API Fine-tuning tak

Class 19 – Class 29 ki mukammal recordings aur handwritten notes se banaya gaya
Interview-ready concept clarity ke liye

Prepared for self-study • Roman Urdu + English technical terms

1. Fine-tuning kya hai aur LLM Training Pipeline
2. Hugging Face Ecosystem
3. Non-Instruction Fine-tuning (Custom Pretraining)
4. Instruction Fine-tuning (SFT)
5. Full Fine-tuning vs PEFT
6. LoRA — Matrix, Rank aur Low-Rank Adaptation (Deep Theory)
7. QLoRA aur Quantization
8. Preference Alignment — RLHF, PPO, GRPO, DPO, ORPO
9. 3-Stage Pipeline aur Model Merging ($M0 \rightarrow M1 \rightarrow M2 \rightarrow M3$)
10. Unsloth Framework
11. Multimodal (Vision) Fine-tuning
12. OpenAI / GPT API Fine-tuning
13. RAG vs Fine-tuning — Real Industry Decision
14. Final Interview Cheat-Sheet (Sab Kuch Ek jagah)

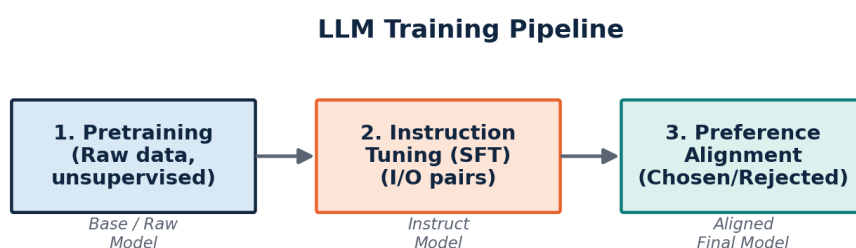
SECTION 1

Fine-tuning kya hai aur LLM Training Pipeline

Fine-tuning ka matlab hai kisi already trained model ko lena aur usay apne domain-specific ya task-specific data par dobara train karna, taake wo us specific use-case mein better perform kare. Yani hum scratch se model nahi banate, balke ek existing model (jaise LLaMA, Mistral, Qwen) ko apni requirement ke mutabiq “customize” karte hain.

LLM Training ke 3 Stages

Har bara LLM (chahe woh Meta ka LLaMA ho ya OpenAI ka GPT) teen stages se guzarta hai:



LLM Training Pipeline: Pretraining → SFT (Instruction Tuning) → Preference Alignment

Stage 1 — Pretraining

Yeh unsupervised / self-supervised learning hai. Model ko internet-scale RAW data (bina kisi label ke) diya jata hai aur usay sirf next-token prediction sikhaya jata hai. Is stage ke baad model ek “raw / base model” banta hai — jaise **meta-llama/Meta-Llama-3-8B**. Yeh model bohat sara general knowledge rakhta hai lekin “instructions follow” karna nahi jaanta.

Stage 2 — Supervised Fine-tuning / Instruction Tuning (SFT)

Ab model ko Input ↔ Output pairs (supervised data) diya jata hai taake wo instructions follow karna seekhe — yani conversational/chatbot/QA banna seekhe. Is stage ke baad model banta hai jaise **meta-llama/Meta-Llama-3-8B-Instruct**.

Stage 3 — Preference Tuning / Preference Alignment

Is stage mein model ko human preferences ke sath align kiya jata hai — yani do responses mein se model ko sikhaya jata hai kaunsa response “behtar / zyada acceptable” hai. Yeh RLHF, DPO waghera techniques se hota hai. Example: **OpenRLHF/Llama-3-8b-rlhf-100k**.

Custom Fine-tuning kya hota hai?

Aap in teeno stages mein se kisi bhi stage ke model ko base bana kar apna khud ka fine-tuning shuru kar sakte hain: apni khud ki SFT (instruction tuning) ya apni khud ki preference tuning. Yeh flexible hai — zaroori nahi ke aap teeno stages hi sequentially

karein, aap kahin se bhi shuru kar sakte hain depending on data availability.

Real-life Decision Rule (Bohat Important)

Agar aapka use-case sirf text/data generation hai (conversational nahi) — jaise pharma company molecule documents par text generate karwana — to RAW/pretrained model ko fine-tune karo. Agar aapka use-case chatbot/QA/conversational AI hai to Instruct model ko fine-tune karo.

Company aur Model Mapping (General Knowledge)

Company	Kya banate hain
Meta	LLaMA models (open-source, USA)
Mistral AI	Mistral models (France)
OpenAI	GPT models (USA, closed-source API)
DeepSeek	DeepSeek models (China) — GRPO technique inhone popularize ki
Sarvam AI	India-based, open models par LoRA/DoRA adapters train karte hain
Alibaba	Qwen models (open-source, multimodal-capable)
Google	Gemma (open) aur Gemini (closed API)

Interview Prep Corner

Interview Sawal	Jawab (Chota aur Clear)
Fine-tuning ki definition batao?	Already trained model ko lena aur usay domain/task-specific data par further train karna taake specific use-case mein performance improve ho.
LLM training ke kitne stages hain?	3 stages: (1) Pretraining – unsupervised, raw data, next-token prediction. (2) SFT/Instruction tuning – supervised, I/O pairs. (3) Preference tuning – human preference alignment (RLHF/DPO).
Raw model vs Instruct model mein kya farq hai?	Raw model sirf next-token predict karta hai, instructions follow nahi karta. Instruct model SFT ke baad conversational ban jata hai aur instructions follow kar sakta hai.

SECTION 2

Hugging Face Ecosystem

Hugging Face (HF) ek AI company hai jo poora ecosystem provide karti hai AI development ke liye: training LLMs from scratch, fine-tuning, evaluation, aur applications hosting. Aaj kal ki puri modern LLM fine-tuning industry ka workflow HF ecosystem ke around ghoomta hai.

Yeh ecosystem ka layout neeche table mein saaf tarah se diya gaya hai.

Native HF Libraries (Musts for LLM Fine-tuning)

- Transformers — model architectures load/use karne ke liye (AutoModel, AutoTokenizer)
- Datasets — dataset load/manage karne ki library
- Tokenizers — text ko tokens mein todne ke liye
- PEFT — Parameter-Efficient Fine-Tuning (LoRA, QLoRA, DoRA implement karne ke liye)
- Accelerate — multi-GPU / distributed training ke liye
- TRL (Transformer Reinforcement Learning) — SFTTrainer, DPOTrainer, PPOTrainer, RewardTrainer
- Bitsandbytes — 4-bit/8-bit quantization ke liye (technically outsider but tightly integrated)
- Safetensors — model weights ko safe format mein save karna
- Evaluate — accuracy, BLEU, ROUGE, perplexity jaise evaluation metrics

HF Hub ke Components

Term	Matlab / Explanation
HF Hub	GitHub jaisa — koi bhi apna model upload kar sakta hai
Dataset Hub	Companies/universities/community ke pre-uploaded datasets
HF Spaces	Apni AI app host karne ki jagah
HF Enterprise	Paid tier — yehi HF ka revenue model hai (core library open-source hai)

Poora Fine-tuning Pipeline (12 Steps — Yeh Bohat Important Hai Interview Ke Liye)

1. Dataset Library / Custom Dataset
2. Preprocessing / Cleaning
3. Tokenizer (Tokenized Dataset)
4. Transformers — Model Loading
5. PEFT (LoRA Adapters attach karna)

- 6. Bitsandbytes (4-bit Quantization)
- 7. Accelerate (Distributed / Multi-GPU Training)
- 8. Trainer / TRL (SFT, RLHF, DPO, PPO, GRPO)
- 9. Evaluation (Accuracy, BLEU, ROUGE, Perplexity, Benchmarks)
- 10. Save Checkpoints (Safetensors / GGUF / GGML)
- 11. Push to Hugging Face Hub
- 12. Inference / Deployment (TGI, TEI, Inference Endpoints) → phir Monitoring & Iteration

Dataset Formats

Term	Matlab / Explanation
Small data	CSV, XLSX, TSV, JSON
Big data (binary)	Parquet (HF ka default), RC, ORC
Instruction format	Alpaca format — instruction / input / output columns
Preference format	Chosen / Rejected columns (jaise Anthropic HH-RLHF dataset)

Tokenization — Text ko Tokens mein Kaise Todte Hain

Transformer pipeline: Training → Parallel → Token → Token IDs → Word Embedding → Positional Encoding → Self-Attention → FFNN

Algorithm	Kaun use karta hai
BPE (Byte Pair Encoding)	GPT models use karte hain. Frequent character pairs ko merge karte hain.
WordPiece	BERT/Google use karta hai. Language model likelihood maximize karta hai.
SentencePiece	T5, LLaMA, Mistral use karte hain. Directly subwords seekhta hai, WordPiece ka updated version.

Example: “I love AI” → [“I”, “love”, “AI”] → Token IDs: [40, 1842, 9552]

BERT Architecture (Numbers Yaad Rakhna Interview Ke Liye)

AutoModel raw encoder load karta hai jiska output sirf “process vector / hidden states” hota hai (koi prediction nahi, kyunke koi “head” attach nahi hai). Prediction lene ke liye ek “head” add karni parti hai upar.

Model	Specs
BERT-Base	12 Transformer/Encoder layers, 768 hidden units, 12 attention heads, 110M parameters

Model	Specs
BERT-Large	24 layers, 1024 hidden units, 16 attention heads, 340M parameters

AutoModel raw encoder load karta hai → output sirf “process vector / hidden states” hota hai, koi prediction nahi (kyunke koi “head” nahi hai). AutoModelForMaskedLM BERT-style masked prediction ke liye, aur AutoModelForCausalLM GPT-style text generation ke liye (yehi modern LLM fine-tuning mein sab se zyada use hota hai).

Unsloth ki Pehli Jhalak (Full Detail Section 10 mein)

HF ka standard workflow slow, VRAM-heavy, aur expensive hota hai. Isi problem ko solve karta hai Unsloth — ek ultra-optimized layer jo HF ke upar bana hota hai. (Poori detail aur diagram Section 10 mein hai.)

Interview Prep Corner

Interview Sawal	Jawab (Chota aur Clear)
PEFT, TRL, Accelerate ka kaam kya hai?	PEFT = LoRA/QLoRA/DoRA implement karta hai. TRL = SFT/DPO/PPO/Reward trainers deta hai. Accelerate = multi-GPU/distributed training manage karta hai.
BPE aur WordPiece mein farq?	BPE frequent character-pairs merge karke tokens banata hai (GPT use karta hai). WordPiece language-model likelihood maximize karke tokens choose karta hai (BERT use karta hai).
Parquet format kyun use hota hai big data ke liye?	Parquet ek binary, columnar storage format hai jo bade datasets ko efficiently store/read karta hai — isliye HF ka default storage format hai.

SECTION 3

Non-Instruction Fine-tuning (Custom Pretraining)

Jab hamare paas sirf RAW data hota hai (PDF, DOCX, PPT, plain text) — jismein koi question-answer pairs nahi hote — to us par Non-Instruction Fine-tuning ki jati hai. Isay Custom Pretraining bhi kaha jata hai, kyunke yeh conceptually pretraining jaisa hi hai (sirf next-token prediction, no I/O pairs).

Neeche pipeline ke saare steps detail mein diye gaye hain.

Real Example: Pharma Company Use-case

Ek pharma company (“Gangadhar” character example) ke paas apne molecule/drug documents (PDFs) hain. Woh chahte hain apna khud ka domain-specific model. Options: (a) scratch se train karo — bohat expensive, (b) existing model ko fine-tune karo, (c) RAG banao. Fine-tuning ka route choose kiya jata hai jab company apna khud ka permanently-customized model chahti ho.

Poora Pipeline — Step by Step

- 1. Raw Data — Pharma PDFs
- 2. Data Parsing — PyMuPDF (fitz) se PDF se text nikalna, page-wise
- 3. Cleaning — Unicode NFKC normalization, zero-width characters hatana, hyphenated line-breaks fix karna, extra spaces, standalone page-numbers hatana — yeh step “very subjective” hai, AI (Claude/ChatGPT) se cleaning code generate karwa sakte hain
- 4. Final Data — paragraphs mein split karna, chote paragraphs (jaise <80 characters) filter out karna
- 5. AI-compatible Format — JSONL mein save karna (reproducibility/compliance ke liye), phir **Dataset.from_list()** se HF Dataset banana
- 6. Train/Test Split — jaise 7 train / 2 validation rows
- 7. Preprocessing (Tokenization) — AutoTokenizer load karna, EOS token ko pad token set karna. group_texts function saare tokens ko concatenate karke fixed block_size (512) mein chunk karta hai — yeh padding nahi hai, balke same-length training blocks banane ka tareeqa hai (GPU utilization behtar hoti hai)
- 8. Model Load karna — 4-bit Quantization (BitsAndBytesConfig) ke sath AutoModelForCausalLM, phir prepare_model_for_kbit_training
- 9. Fine-tuning of Model (PEFT) — LoraConfig + get_peft_model — sirf ~1% parameters trainable hote hain 1B model ke total parameters mein se
- 10. Save / Push to HF Hub — LoRA adapter + tokenizer save karna
- 11. Inferencing — PeftModel.from_pretrained() se adapter attach karna, ya merge_and_unload() se permanently base model ke sath fuse karna

Labels = Input IDs (Causal LM ke liye)

Non-instruction fine-tuning mein labels simply input_ids ki copy hoti hain. Hugging Face internally shift karta hai taake model next-token prediction seekhe. Padded positions ke labels -100 set kiye jate hain taake PyTorch loss calculation mein unhe ignore kare.

Do Tarike Block/Padding Handle Karne Ke

Approach	Detail
Custom group_texts (Industry style)	Saare tokens concatenate karke fixed 512-token blocks banate hain — koi padding waste nahi hoti, GPU utilization behtar. Yeh approach Class 22/23 mein use hui.
Simple tokenizer padding	padding='max_length', truncation=True, max_length=512 — simple lekin GPU cycles padding tokens par waste hote hain.

Result: Kya Milta Hai Non-Instruction FT Se?

Model domain knowledge/vocabulary seekh leta hai (jaise pharma terminology, patterns), lekin abhi bhi conversational nahi banta — woh sirf text generate karta hai, questions ka jawab nahi de sakta. Isi liye agle stage Instruction Fine-tuning ki zaroorat hoti hai.

Interview Prep Corner

Interview Sawal	Jawab (Chota aur Clear)
Non-instruction fine-tuning kab use hoti hai?	Jab hamare paas RAW documents (PDF/DOCX/PPT) hon aur hum chahte hon model domain-specific knowledge/language absorb kare, but conversational banana zaroori na ho.
group_texts function kya karta hai?	Saare tokens ko ek line mein concatenate karke fixed-size (jaise 512) blocks mein todta hai — taake har training example same length ka ho aur GPU padding waste na ho.
Labels kaise banti hain causal LM training mein?	Labels = input_ids ki copy. HF internally shift karta hai taake model next-token predict karna seekhe. Padding positions ke labels -100 hoti hain taake loss mein exclude ho.

SECTION 4

Instruction Fine-tuning (SFT)

Instruction Fine-tuning ka maqsad hai model ko conversational banana — yani woh questions samajh kar unka proper jawab de sake, sirf text generate na kare.

Neeche teeno real-world methods detail mein diye gaye hain.

Real-time mein Instruction Data Kahan se Aata Hai?

Method 1: Human-created Q&A Data

Domain experts manually questions aur answers banate hain. Example format:

```
{  
  "instruction": "Explain the mechanism of action of Metformin.",  
  "input": "",  
  "output": "Metformin primarily activates AMPK, reduces hepatic gluconeogenesis, and improves glucose uptake."  
}
```

Method 2: Existing Customer/Support Conversations

Companies ke paas already real conversations hoti hain — customer questions, doctor/agent answers, support tickets, chat transcripts, email threads. Inko clean karke instruction format mein convert kiya ja sakta hai.

Method 3: Synthetic Data Generation

Ek strong LLM use karke raw documents se Q&A pairs generate karwaye jate hain:

- Raw pharma document
- ↓ LLM questions aur answers generate karta hai
- ↓ Human/domain expert review karta hai
- ↓ Clean instruction dataset
- ↓ Instruction fine-tuning

Regulated Domains Mein Zaroori Baat

Pharma ya healthcare jaisi regulated domains mein synthetic data ko hamesha domain expert se review karwana chahiye, warna wrong/hallucinated information model mein permanently seekh ja sakti hai.

Data Format — Alpaca Format

Sab se common format hai Alpaca format: instruction / input / output columns. In teeno ko concatenate karke ek single “text” string bana di jati hai training ke liye:

Alpaca Field	Kya hai
instruction	Kya karna hai (jaise “Summarize the output”)

Alpaca Field	Kya hai
input	Agar koi extra context/input diya gaya ho (empty ho sakta hai)
output	Model ka expected answer
Combined text	"instruction# ---- input# ---- output#" — ek single training string

System / User / Assistant Roles (OpenAI Format)

Term	Matlab / Explanation
system	Instruction/behavior setting — jaise “You are a helpful medical assistant”
user	User ka actual sawal
assistant	Model ka expected jawab

Agar system message hata di jaye to model phir bhi jawab de sakta hai, lekin uska behavior control chala jata hai — woh company-specific tone/rules follow nahi karega.

Interview Prep Corner

Interview Sawal	Jawab (Chota aur Clear)
Alpaca format kya hai?	Ek instruction-tuning data format jismein teen fields hoti hain: instruction, input, output. Inhe combine karke ek text string banti hai jo model ko train karti hai.
Instruction data ke real-world sources kya hain?	(1) Human/domain experts manually banate hain, (2) existing support tickets/chats se convert karte hain, (3) strong LLM se synthetic generate karke expert se review karwate hain.
System message ka role kya hai?	System message model ka overall behavior/tone/rules set karta hai — jaise “only smartphone-related queries answer karo”. Isay hata dein to model kaam to karega lekin uska behavior control loose ho jayega.

SECTION 5

Full Fine-tuning vs PEFT

Full-Parameter Fine-tuning

- Model ke saare weights update hote hain
- 7B model ka matlab hai 7 billion weights — 13B model ka 13 billion, waghera
- Saare weights repeatedly update hote hain multiple epochs ke liye
- Storing aur updating weights ke liye bohat zyada memory chahiye
- Sirf large GPUs ya GPU clusters par possible hai — expensive training

Parameter-Efficient Fine-tuning (PEFT)

- Saare weights update nahi hote — original weight matrices freeze (frozen) rehti hain
- Sirf ek chota number of additional parameters train hota hai

PEFT Achieve Karne Ke Tareeqe

Method	Kya karta hai
LoRA (Low-Rank Adaptation)	Sab se popular; small low-rank matrices train karta hai instead of full weights → QLoRA iska quantized version hai
DoRA (Weight-Decomposed LoRA)	Weight ko magnitude + direction mein decompose karta hai, LoRA se thora advanced
Prefix Tuning	Learnable prefix vectors add karta hai
Prompt Tuning	Sirf input embeddings seekhta hai
BitFit	Sirf bias parameters train karta hai
IA ³	Attention mechanisms ko scale karta hai

LoRA ke Fayde

Less memory + Faster training + Lower cost + Easy customization. Single GPU ya consumer hardware par bhi fine-tuning enable karta hai. Performance almost full fine-tuning jaisi hi rehti hai.

Interview Prep Corner

Interview Sawal	Jawab (Chota aur Clear)
Full fine-tuning kyun expensive hai?	Kyunke model ke saare billions of weights update hote hain, jinhe store aur update karne ke liye huge memory/GPU clusters chahiye hote hain.

Interview Sawal	Jawab (Chota aur Clear)
PEFT ka core idea kya hai?	Original model weights ko freeze kar dete hain aur sirf thora sa additional parameters train karte hain — isse memory/cost drastically kam ho jate hain, aur performance almost same rehti hai.

SECTION 6

LoRA — Deep Theory (Matrix, Rank aur Low-Rank Adaptation)

Yeh section interview ke liye sab se important hai kyunke LoRA sab se zyada asked concept hai. Isay step by step, math ke sath samjhte hain.

Weights Kya Hote Hain?

Weight = importance score. Ek neural network yeh decide karta hai ke “kaunsa feature zyada important hai”. Example: **Taste = Patty×0.9 + Cheese×0.7 + Sauce×0.5 + Bun×0.2** — yahan 0.9, 0.7, 0.5, 0.2 weights hain jo batate hain har ingredient ka taste par kitna asar hai.

Training ke Doran Kya Hota Hai?

- Shuru mein weights random hote hain
- Training ke doran: Loss calculate hoti hai, phir weights gradients se update hoti hain (Back Propagation)
- Goal: best weights dhoondna jo minimum loss dein

Transformer Architecture Mein Weights Kahan Hain?

Term	Matlab / Explanation
W_Q (Query)	Linear layer weight matrix — attention ke liye Query banata hai
W_K (Key)	Linear layer weight matrix — Key banata hai
W_V (Value)	Linear layer weight matrix — Value banata hai
W_O (Output)	Attention output ko wapas combine karne wali weight matrix
Attention Weights	Sab se important — yeh decide karte hain ke ek word doosre word ke liye kitna important hai

Example: “I love machine learning” — jab model “learning” word process kar raha hota hai, to woh decide karta hai ke “machine” word zyada important hai ya “love”. Yeh importance hi attention weights hoti hai.

Deep Learning = A Matrix Game

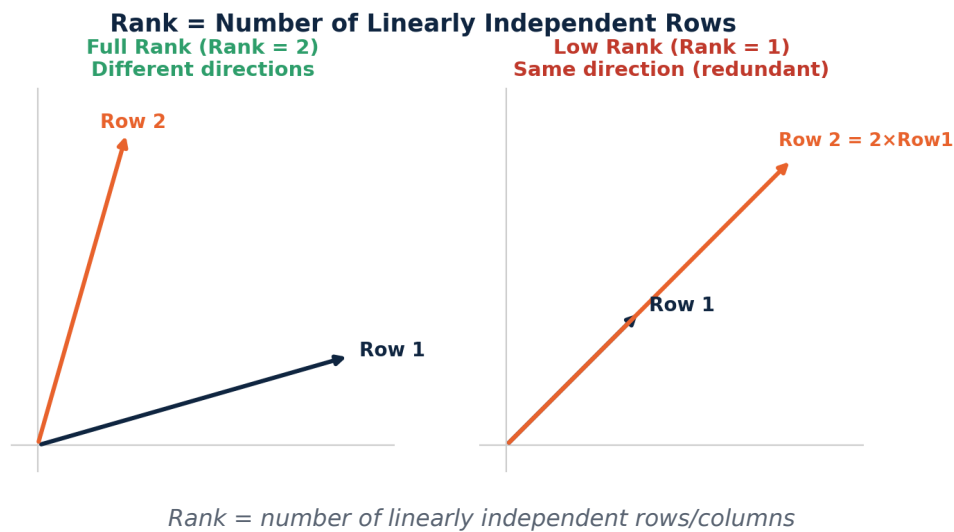
Output = Input × Weights + Bias. Input → vector, Weights (model parameters) → matrix, Output → transformed vector. Images → matrices (pixels), Videos → 3D matrices, Text Embeddings → vectors. Attention = matrix multiplication (Q, K, V → matrices).

Matrix Basics (Scalar, Vector, Matrix)

Type	Example
Scalar	Sirf ek number — jaise 5
Vector	Numbers ki ek line — jaise [1, 2, 3]
Matrix	Numbers ka table/grid — jaise [[1,2],[3,4]]. Matrix = vectors ka collection. Vector = matrix ka special case (1 row ya 1 column)

2×2 matrix ka matlab hai 2 rows, 2 columns. $m \times n$ matrix ka matlab m rows aur n columns.

Rank Kya Hai? (Interview Mein Bohat Poocha Jata Hai)

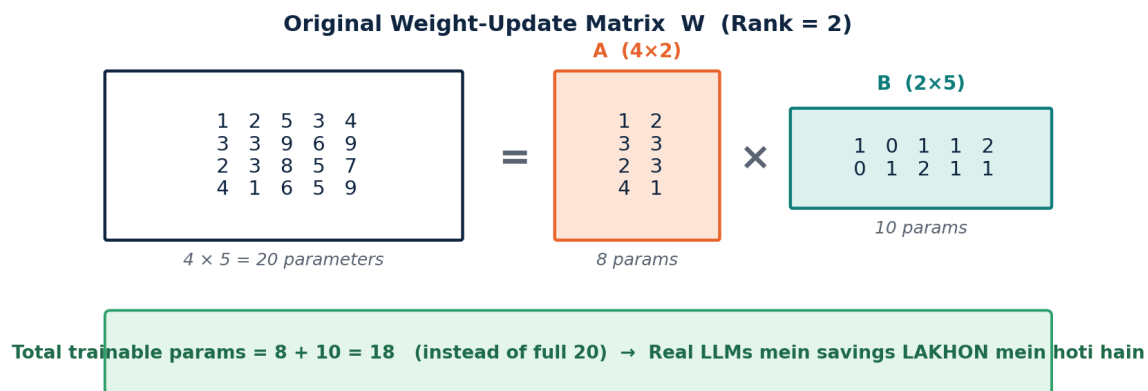


Rank = kitni rows (ya columns) linearly independent hain, yani kitni “unique/new information wali” hain jo doosri row se derive nahi ho sakti.

Concept	Real-world analogy
Full Rank	1000 log jinki sab ki alag personality hai — koi bhi doosre se predict nahi ho sakta
Low Rank	Sirf 2-3 types ke log hain, baaki sirf scaled versions hain — information repeat ho rahi hai

General rule: $m \times n$ matrix ka maximum rank = $\min(m, n)$. Agar ek row doosri row ka multiple hai (jaise $\text{Row 2} = 2 \times \text{Row 1}$), to woh information repeat ho rahi hai — rank low hoga.

LoRA ka Core Math Example (Yeh Numeric Example Interview Mein Bohat Kaam Aata Hai)

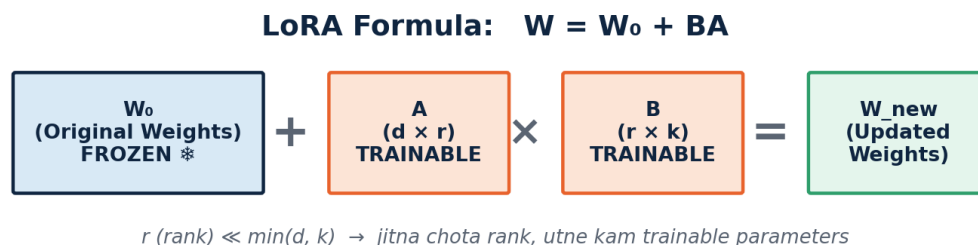


4×5 matrix (20 parameters) ko Rank=2 par decompose kar ke sirf 18 parameters mein represent kiya

Ek 4×5 matrix ke actual 20 parameters hain. Agar iska rank sirf 2 hai (yani sirf 2 columns hi independent hain, baaki unka linear combination hain), to hum isay do chote matrices mein decompose kar sakte hain: A (4×2) aur B (2×5).

A ke parameters = $4 \times 2 = 8$, B ke parameters = $2 \times 5 = 10$. Total = $8 + 10 = 18$ parameters — jo already original 20 se kam hain is chote example mein. Real LLMs mein jahan matrices hazaron dimensions ki hoti hain, yeh saving massive hoti hai (millions se lakhs tak kam ho jate hain trainable parameters).

LoRA Formula



LoRA: $W = W_0 + BA$. W_0 (original weight) frozen rehta hai, sirf A aur B (adapter) trainable hain

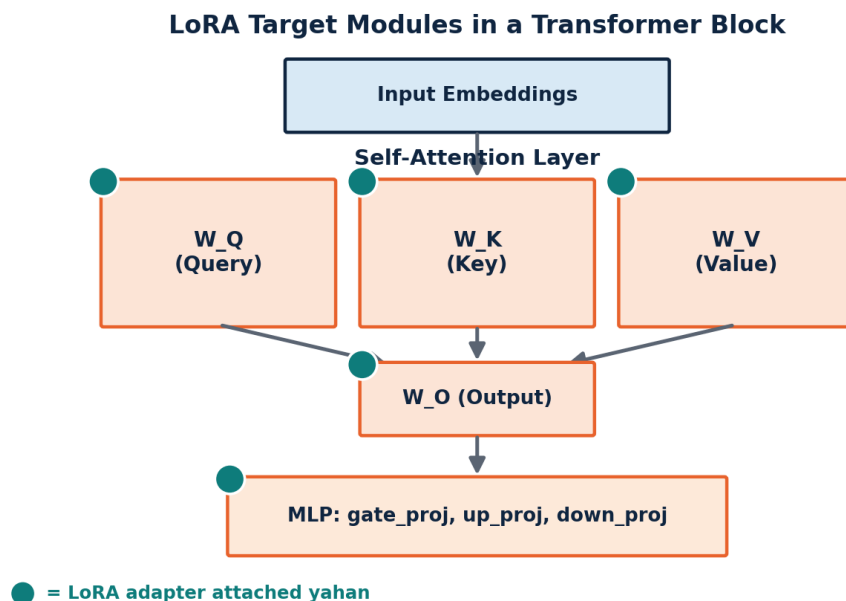
LoRA ka main formula: $W = W_0 + BA$

Symbol	Matlab
W_0	Original pretrained weight matrix — frozen (train nahi hota)
A, B	Do choti low-rank matrices — trainable (LoRA Adapter)
r (rank)	Bohat chota hota hai ($r \ll d, k$) — jitna chota rank, utne kam parameters

LoRA ka assumption yeh hai: “weight update matrix full rank nahi hota, uska rank chota ho sakta hai”. Isi wajah se hum bade weight-update matrix ko do chote matrices ke multiplication se approximate kar sakte hain — ise low-rank update kehte hain.

Training update: $W_{\text{new}} = W_0 + BA$, jismein sirf B aur A (adapter) AdamW optimizer se update hoti hain, W_0 hamesha frozen rehta hai.

LoRA Kahan Apply Hoti Hai LLM Mein?



LoRA adapters attention (W_Q , W_K , W_V , W_O) aur MLP layers dono par attach hoti hain

Original LoRA research paper mostly LoRA W_q aur W_v (query & value) par apply karta hai. Paper ke experiments dikhate hain: sirf W_q akela → average performance, sirf W_v akela → average, lekin $W_q + W_v$ dono sath → best overall performance.

Practical/Industry-default rule aaj kal: **target_modules = ["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"]** — yani attention layers (Q,K,V,O) ke sath sath FFN/MLP layers (gate, up, down projections) par bhi LoRA apply ki jati hai.

Research Paper Benchmark (GPT-3 Example)

Method	Trainable Parameters
GPT-3 (Full Fine-tuning)	175,255.8 Million trainable parameters
GPT-3 (LoRA)	Sirf 4.7 Million trainable parameters — phir bhi WikiSQL/MNLI/SAMSum benchmarks par comparable ya behtar accuracy

Interview Prep Corner

Interview Sawal	Jawab (Chota aur Clear)
LoRA ka core idea one line mein batao?	Original weight matrix ko freeze kar dete hain aur ek chota, low-rank update (do chote matrices A aur B ka multiplication) train karte hain — isse trainable parameters drastically kam ho jate hain.

Interview Sawal	Jawab (Chota aur Clear)
Rank kya represent karta hai?	Rank batata hai matrix mein kitni linearly independent (unique) rows/columns hain — yani kitni actual unique information hai. Low rank = information repeat/redundant.
LoRA formula likho.	$W = W_0 + BA$, jahan W_0 frozen hai aur B, A trainable low-rank matrices hain.
LoRA kaunse layers par sab se zyada apply hoti hai?	Attention ki Q aur V projection matrices (W_q, W_v) par sab se zyada effective hoti hai; industry mein aaj kal Q,K,V,O + MLP (gate/up/down proj) sab par bhi apply ki jati hai.

SECTION 7

QLoRA aur Quantization

QLoRA = Quantization + LoRA. Isay “LoRA 2.0” bhi kaha ja sakta hai — yeh base model ke weights ko compress (quantize) kar deta hai, jabke LoRA adapters trainable rehte hain.

Precision → Memory Trade-off (per parameter)



7B model: FP16 $\approx$ 14 GB $\rightarrow$ 4-bit (QLoRA) $\approx$ 3.5-5 GB

Precision se Memory ka trade-off — FP32 $\rightarrow$ FP16 $\rightarrow$ INT8 $\rightarrow$ INT4 (QLoRA)

Quantization Kya Hai?

Quantization model weights ki numerical precision compress karna hai — high-precision numbers (jaise float32) ko lower-precision (jaise int8, int4) mein convert karna, taake memory kam lage. Example: 32.885 (float32, 4 bytes) $\rightarrow$ 32 (int8, 1 byte).

Precision Type	Memory
FP32	32 bits = 4 bytes per parameter (Full Precision)
FP16	16 bits = 2 bytes per parameter (Half Precision)
INT8	8 bits = 1 byte per parameter
INT4 (4-bit)	4 bits = 0.5 bytes per parameter

Example: 7B model FP16 mein $\approx$ 14 GB memory leta hai, lekin 4-bit mein $\approx$ 3.5-5 GB (overhead ke sath) — yani almost 3-4x kam memory.

LoRA vs QLoRA Comparison

Feature	LoRA	QLoRA
Model Precision	LoRA: FP16/FP32	QLoRA: 4-bit (quantized)
Memory Usage	LoRA: Medium	QLoRA: Very Low
Speed	LoRA: Fast	QLoRA: Slightly slower
GPU Requirement	LoRA: 16-24 GB	QLoRA: 8-16 GB enough
Library	LoRA: PEFT	QLoRA: PEFT + bitsandbytes

QLoRA Kaise Kaam Karta Hai (2 Ideas Combine)

1) Quantization — base model weights ko 4-bit mein compress karna. 2) LoRA — trainable low-rank adapters upar attach karna. Is combination se “recoverable quantization” milti hai — kam memory use hoti hai lekin fine-tuning effective rehti hai.

Interview Prep Corner

Interview Sawal	Jawab (Chota aur Clear)
Quantization kya karti hai?	Model weights ki numerical precision (jaise FP32) ko lower precision (jaise INT4/INT8) mein compress karti hai, taake memory footprint kam ho.
QLoRA aur LoRA mein basic farq?	LoRA FP16/FP32 precision par kaam karta hai, medium memory chahiye. QLoRA base model ko 4-bit quantize kar deta hai, isliye bohat kam GPU memory (8-16GB) mein bhi fine-tuning ho jati hai.

SECTION 8

Preference Alignment — RLHF, PPO, GRPO, DPO, ORPO

Preference Tuning Kya Hai?

Preference tuning model ko human responses/preferences ke sath align karti hai. Data format: Prompt, Chosen (behtar jawab), Rejected (bura jawab).

```
{"prompt": "Explain the primary mechanism of action of metformin.", "chosen": "Metformin primarily acts by activating AMP-activated protein kinase (AMPK)...", "rejected": "Metformin mainly works by increasing insulin secretion..."}
```

4 Methods — Evolution Timeline

Method	Kya Khaas Hai
1. RLHF (OpenAI)	PPO (Proximal Policy Optimization) use karta hai — Reinforcement Learning based. Complex, 4 models chahiye.
2. GRPO (DeepSeek)	PPO ka updated version, value model ko drop kar deta hai. Still RL-based.
3. DPO (Stanford, NeurIPS 2023)	Non-RL, simple supervised technique. Sirf 2 models chahiye (policy + reference).
4. ORPO (KAIST AI)	Non-RL, advanced DPO variant, aur zyada simplified.

Important Industry Reality

RLHF ab “classical/largely outdated” mana jata hai practice mein — foundational understanding aur interview ke liye zaroori hai, lekin aaj kal industry mein DPO aur ORPO hi actually use hote hain kyunke woh mechanically bohat simple hain.

RLHF ke 3 Stages (Deep Dive)

Stage 1: Data Preparation

Human preference data collect karna (prompt + chosen + rejected) — wahi mechanism jo ChatGPT ke “which response do you prefer” UI mein hota hai. Reference dataset: Anthropic ka HH-RLHF.

Stage 2: Reward Model Training

Ek LLM ko sequence classification head (1 output label = scalar reward score) ke sath train karte hain (prompt, chosen, rejected) triples par, taake woh chosen ko rejected se zyada score de sake. Yeh reward model phir “LLM-as-judge” ban jata hai, human judgment ka proxy.

Stage 3: PPO (Reinforcement Learning Step)

PPO stage mein 4 models use hote hain:

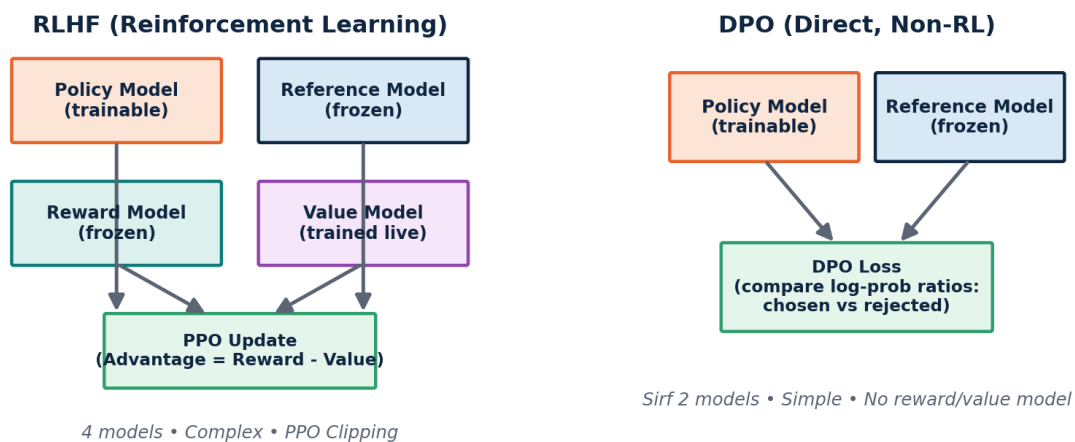
Term	Matlab / Explanation
Policy Model (Actor)	Woh actual LLM jo train ho raha hai/preference-align ho raha hai
Reference Model	Policy model ki frozen copy (pretraining snapshot) — sirf KL divergence measure karne ke liye, khud train nahi hota
Reward Model	Stage 2 mein already trained, frozen — actual reward deta hai
Value Model (Critic)	PPO ke doran hi train hota hai, expected reward predict karta hai

Advantage score = Actual reward (reward model se) – Expected reward (value model se). PPO phir policy ke weights update karta hai taake high-advantage responses ki probability barhe, using PPO clipping (ratio of new/old action probabilities ko $[1-\epsilon, 1+\epsilon]$ range mein rakhta hai, jaise $\epsilon=0.2$) taake destructively bade updates na hon.

KL Divergence/Penalty: yeh measure karta hai ke policy model apne original (reference) behavior se kitna “drift” kar gaya hai — taake model bilkul random/unrelated responses dena shuru na kar de.

DPO — Direct Preference Optimization (Simple aur Modern)

- Non-RL hai — koi reward model, koi value model, koi PPO clipping nahi chahiye
- Sirf 2 models use hote hain: trainable policy model aur frozen reference model
- Dono models chosen aur rejected responses ke liye log-probabilities compute karte hain
- DPO ka implicit reward formula compare karta hai: $\log P_{\text{policy}}(\text{chosen}) / \log P_{\text{policy}}(\text{rejected})$ ratio, reference model ke isi ratio ke against
- Loss kam hoti jati hai jaise-jaise policy model chosen ko rejected se zyada prefer karne lagta hai (reference ke mukable mein)
- Yeh simplicity hi wajah hai ke DPO/ORPO ne RLHF/PPO ko largely replace kar diya hai



RLHF (4 models, RL-based) vs DPO (2 models, Non-RL, simple direct loss)

TRL Library mein Implementation

TRL Class	Kis liye
RewardTrainer / RewardConfig	Reward model train karne ke liye (RLHF Stage 2)
PPOTrainer / PPOConfig	PPO training ke liye (4 models pass karte hain)
DPOTrainer / DPOConfig	DPO training ke liye (koi external reward model nahi chahiye)

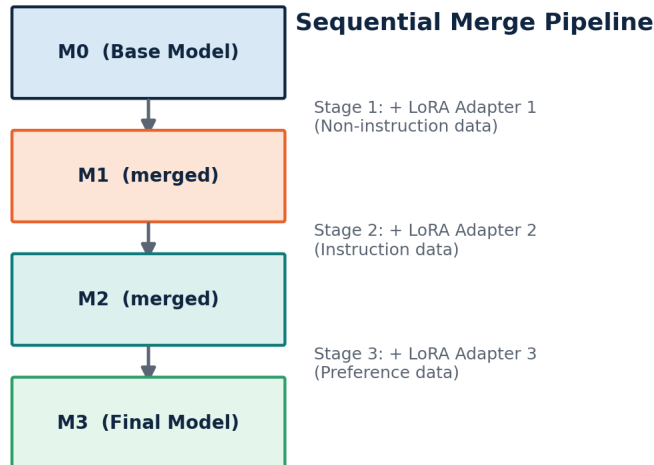
Interview Prep Corner

Interview Sawal	Jawab (Chota aur Clear)
RLHF aur DPO mein basic farq kya hai?	RLHF Reinforcement Learning based hai, 4 models (policy, reference, reward, value) chahiye, PPO algorithm use karta hai. DPO non-RL, simple supervised technique hai, sirf 2 models (policy + frozen reference) chahiye, direct loss function se train hota hai.
PPO clipping kyun zaroori hai?	Taake policy ke weights mein ek hi step mein bohat bada/destructive update na ho jaye — yeh training ko stable rakhta hai.
Aaj kal industry mein kaunsi technique zyada use hoti hai?	DPO aur ORPO — kyunke woh RLHF/PPO se mechanically bohat simpler hain aur similar ya behtar results dete hain.
Reward model kya kaam karta hai?	Yeh ek trained classifier hai jo (prompt, response) ko dekh kar ek scalar reward score deta hai — yeh human judgment ka proxy/LLM-as-judge ban jata hai.

SECTION 9

3-Stage Pipeline aur Model Merging (M0 → M1 → M2 → M3)

Yeh section poore fine-tuning course ka sab se confusing lekin sab se important concept clear karta hai — ke teeno stages ko sequentially kaise combine karte hain.



Correct Approach: har stage ka adapter merge hota hai agle stage ke base model mein

Model Naming Convention

Term	Matlab / Explanation
M0	Raw / Base model (jaise TinyLlama) — bilkul shuru wala model
Stage 1	M0 + LoRA Adapter 1 (non-instruction/raw data par train) → merge → M1
Stage 2	M1 (naya base) + LoRA Adapter 2 (instruction data par train) → merge → M2
Stage 3	M2 (naya base) + LoRA Adapter 3 (preference/DPO data par train) → merge → M3 = Final Model

Analogy Yaad Rakhne Ke Liye

M0 = “Class 10 ka student”. LoRA Adapter 1 = “Class 11 ka knowledge” (isse M1 = “Class 10+11” banta hai). LoRA Adapter 2, jo M1 par train hua, us mein “Class 12 ka knowledge” hai. Agar aap Adapter 2 ko seedha M0 (sirf Class 10 wale) par laga dein — Class 11 skip karke — to result broken/incoherent hoga, kyunke Adapter 2 sirf M1 ke weights ke context mein hi meaningful hai.



Append = temporary (runtime/VRAM) vs Merge = permanent (weights fuse ho jate hain)

Do Valid Approaches

Term	Matlab / Explanation
1. Sequential Merge Approach (Industry Standard)	Har stage ka adapter merge hota jata hai growing base model mein, agla stage usi par train hota hai. Result: ek hi final model jismein saari 3 stages ki capability combined hai. Yeh assume kiya jata hai ke ChatGPT-jaise models isi tarah bante hain.
2. Isolated/Parallel Adapters Approach	Teeno adapters ko alag-alag, sab ke sab same base model M0 par train karo, aur unhe separately rakho. Inference time par jo bhi task ho, uske hisaab se adapter load karo (task-based routing). Har adapter mein sirf apni hi stage ka knowledge hoga — cumulative nahi. Kam use hota hai kyunke ek combined model behtar answers deta hai.

Append vs Merge

Term	Matlab / Explanation
Append (Runtime/VRAM)	<code>PeftModel.from_pretrained()</code> — adapter temporarily attach hota hai, quick inference/testing ke liye. Base model ke weights change nahi hote, permanent nahi hai.
Merge (Permanent)	<code>merge_and_unload()</code> — adapter weights permanently base model ke weights mein fuse ho jate hain. Dono ka inference result same hota hai, sirf storage/workflow ka farq hai.

Important: Merging se model ke parameters ka count barhta nahi — purane weights replace/update hote hain, add nahi hote.

Flexibility

Aap non-sequentially bhi jump kar sakte hain data availability ke hisaab se — jaise agar sirf conversational data hai to non-instruction stage skip kar ke seedha M0 se preference tuning tak bhi ja sakte hain. Yeh rigid nahi hai. Lekin ek base model par ek waqt mein sirf ek hi adapter load ho sakta hai — multiple adapters ek sath simultaneously load karne se error aata hai.

Interview Prep Corner

Interview Sawal	Jawab (Chota aur Clear)
Agar aapke paas 3 separate LoRA adapters hon (non-instruction, instruction, preference), to unhe final model banane ke liye kaise combine karenge?	Sequential merge approach: Adapter 1 ko M0 par train karke merge karo → M1 banao. Phir Adapter 2 ko M1 par train karke merge karo → M2 banao. Phir Adapter 3 ko M2 par train karke merge karo → final M3 model banao.
Kya hum kisi bhi adapter ko directly M0 (raw base) par apply kar sakte hain?	Nahi, sirf pehli stage ka adapter M0 par directly kaam karega. Baad ke stages ke adapters sirf unke apne training-time base model par hi meaningful hain — unhe skip karke M0 par lagana broken result dega.

SECTION 10

Unsloth Framework

Unsloth Kyun Zaroori Hai?

Standard Hugging Face workflow ke 3 bade issues hain: slow, huge VRAM consume karta hai, aur expensive hai. Inhi problems ko solve karta hai Unsloth — ek ultra-optimized LLM fine-tuning framework jo Hugging Face ecosystem ke upar bana hai.

Unsloth ke Claims

- 2x se 3x faster training
- 50-80% kam VRAM usage
- Bade models ko free GPUs (jaise T4) par bhi train karne ki ability

Framework	VRAM Required
Standard HF	24 GB VRAM
Unsloth	Sirf ~7-8 GB VRAM

GPU (Unsloth)	Max Context Length
12 GB GPU	~21K tokens
16 GB GPU	~40K tokens
24 GB GPU	~78K tokens
80 GB GPU	~340K tokens

Jabke standard HF 80 GB GPU par bhi sirf ~28K tokens support karta hai — itna bara farq is liye hota hai kyunke Unsloth memory ko bohat efficiently manage karta hai.

Yeh Speed/Memory Kaise Milti Hai?

- Custom CUDA kernels aur Triton kernels (Triton ek Python-like GPU language hai jo highly-optimized CUDA kernels generate karti hai)
- Fused Attention aur Flash Attention
- Manual backpropagation, smart checkpointing, aur automatic sequence packing

Sab Se Important Point

Unsloth approximation tricks use nahi karta — yeh exact math maintain karta hai. Isi liye accuracy loss negligible hoti hai aur model quality degrade nahi hoti. Unsloth speed aur memory efficiency improve karta hai bina actual mathematical correctness change kiye.

HF vs Unsloth — Real Benchmark (Class 25 mein khud test kiya gaya)

Test	Result
200 rows data	HF: ~72-94 sec training Unsloth: kam time aur kam VRAM
2000 rows data	HF: ~118 sec Unsloth: sirf ~77 sec — farq data barhne ke sath aur zyada bara hota jata hai

Conclusion: Unsloth training speed aur VRAM dono par jeet ta hai. Industry production fine-tuning ke liye Unsloth ko HF se prefer kiya jata hai.

Unsloth mein Code Kaise Different Hai?

Term	Matlab / Explanation
Model+Tokenizer Load	FastLanguageModel.from_pretrained() — ek hi call mein sab kuch (tokenizer + quantization + model) ho jata hai
LoRA Apply	FastLanguageModel.get_peft_model() — conceptually HF ke get_peft_model() jaisa hi hai
SFT (Stage 1 & 2)	SFTTrainer/SFTConfig (TRL se) — packing=True (non-instruction ke liye), packing=False (instruction ke liye)
DPO (Stage 3)	PatchDPOTrainer() pehle apply karo, phir DPOTrainer/DPOConfig (TRL se)

Yaad Rakhna

Unsloth “just a wrapper” hai Hugging Face ke upar — sab kuch internally HF ke Transformers/PEFT/TRL se hi chal raha hota hai, Unsloth sirf optimized CUDA/Triton kernels inject karta hai speed/memory gains ke liye. Same M0→M1→M2→M3 merge pattern yahan bhi apply hota hai.

Interview Prep Corner

Interview Sawal	Jawab (Chota aur Clear)
Unsloth kya hai aur kyun use karte hain?	Unsloth ek optimized fine-tuning framework hai jo HF ke upar bana hai. Yeh custom CUDA/Triton kernels, Flash Attention aur smart checkpointing use karke 2-3x faster training aur 50-80% kam VRAM deta hai, bina accuracy loss ke.
Kya Unsloth Hugging Face ko replace karta hai?	Nahi, Unsloth HF ko replace nahi karta — yeh HF ecosystem (Transformers, PEFT, TRL) ke upar ek optimized layer/wrapper hai.

SECTION 11

Multimodal (Vision) Fine-tuning

Multimodal Fine-tuning Kya Hai?

Same LLM architecture, lekin ab input sirf text nahi — image + text input → text output. Real use-case: domain-specific vision-capable model banana (jaise product images se description generate karna) instead of general-purpose OCR/vision model par depend karna.

Data Preparation

- Custom dataset: product images (jaise iPhones) + hand-written captions har image ke liye
- Captions accurate hone chahiye — manual effort lagta hai, IDs se properly map karna zaroori hai
- 3 columns ka standard format: image (path ya actual image object), text (caption), aur messages (structured: user role mein text+image, assistant role mein caption)

Image Store Karne Ke 2 Tareeqe

Term	Matlab / Explanation
String Path	Images alag rakhi jati hain (GitHub/Drive/S3 par) — lightweight lekin image folder alag se distribute karna padta hai
Actual Image Object	HF ke Features/Image() datatype se dataset ke andar hi image object save hota hai — self-contained lekin heavy

Real HF datasets examples: LLaVA-Instruct-150K (image names only, COCO se sourced, alag download karni padti hain), LaTeX-OCR, Flickr30k, ChartQA (yeh actual image objects rakhte hain).

Multimodal-Capable Models

Term	Matlab / Explanation
Qwen2-VL / Qwen2.5-Omni	Alibaba
LLaVA / LLaMA-Vision	Meta
Gemma (Vision variants)	Google
Mistral (Vision variants)	Mistral.ai
GPT-5, Gemini 2.5 Flash	OpenAI, Google — closed-source multimodal

Implementation (Unsloth Vision) — Class-based Production Pattern

Class 26 se seekhi gayi ek important practice: production-style code Python classes mein likhna, sirf linear notebook cells mein nahi. Yeh pattern aage RAG/Agent projects mein bhi use hoga.

- VisionFineTuner class: load_model → prepare_dataset → build_trainer → train → save → quick_infer → sab ek run() method se chalte hain
- Config dataclass (@dataclass decorator): sab settings (model name, LoRA rank/alpha/dropout, training args) ek jagah
- model_registry aur data_registry dictionaries: kisi bhi model/dataset ko simple key se swap kar sakte hain, code rewrite nahi karna padta
- Model load: FastVisionModel.from_pretrained(), LoRA apply: FastVisionModel.get_peft_model() (vision layers, language layers, attention, MLP sab par)
- Training: SFTConfig + SFTTrainer (wahi class jo text fine-tuning mein use hoti hai)
- Inference: FastVisionModel.for_inference(), chat template se image+instruction, TextStreamer se output stream

Real-World Reality

Multimodal fine-tuning industry mein relatively rare hai text-only ke mukable mein, lekin concept samajhna zaroori hai. Video/audio fine-tuning aur bhi rare hai — sirf Unsloth docs mein explore karne ke liye chora gaya (out of scope for this course).

Interview Prep Corner

Interview Sawal	Jawab (Chota aur Clear)
Multimodal fine-tuning normal text fine-tuning se kaise different hai?	Data format mein image bhi included hoti hai (image+text input → text output). Model bhi vision-capable hona chahiye (jaise Qwen2-VL, LLaVA). Baaki pipeline (LoRA, SFTTrainer, merge) conceptually same rehta hai.

SECTION 12

OpenAI / GPT API Fine-tuning

Yeh closed-source, API-based fine-tuning hai — koi GPU chahiye nahi, training poori OpenAI ke servers par hoti hai. Aap sirf data + commands API ke zariye submit karte hain.

Requirements

- OpenAI API key (paid)
- Supported models: GPT-4.1, GPT-4.1 Mini/Nano, GPT-4o, GPT-4o Mini, GPT-3.5 Turbo (legacy)
- Data format OpenAI-specific hai (Alpaca nahi) — messages key ke andar system/user/assistant roles
- Assistant field mandatory hai — agar missing ho to “last message must be from the assistant” error aata hai
- Minimum 10 JSONL rows chahiye hoti hain warna validation error aata hai

Poora Flow

- 1. Install OpenAI SDK
- 2. Set API key
- 3. Create OpenAI client
- 4. Test base model
- 5. Load training data
- 6. Upload training file
- 7. Create fine-tuning job
- 8. Check fine-tuned model status
- 9. Use fine-tuned model for inference

Real Demo Numbers (Class 27)

GPT-4.1 Nano ko 10-row smartphone customer-support dataset par fine-tune kiya gaya — poora process ~25-30 minutes laga, aur total cost \$0.10 se kam raha (training rate ~\$1.5/hour hai, plus input/output token costs).

Bohat Important Insight

Fine-tuned model vs base/general-purpose model — is small demo mein dono ne reasonably correct jawab diya (base model bhi theek chala kyunke system message set tha). Isi liye aapko empirically evaluate karna chahiye ke fine-tuning aapke use-case ke liye actually zaroori hai ya sirf prompting/RAG hi kaafi hai.

Azure AI Foundry aur AWS Bedrock par bhi similar managed fine-tuning workflows available hain (same conceptual steps). AWS Bedrock GPT fine-tuning support nahi karta lekin RAG/agentic systems ke liye strong hai.

Interview Prep Corner

Interview Sawal	Jawab (Chota aur Clear)
GPT/OpenAI API fine-tuning ka data format kya hai?	JSONL with 'messages' key containing system, user, assistant roles. Assistant role mandatory hai, warna validation error aata hai.
GPU-based fine-tuning (HF/Unsloth) aur OpenAI API fine-tuning mein basic farq?	HF/Unsloth mein hum khud GPU par model train karte hain aur weights control karte hain (open-source). OpenAI API fine-tuning mein training closed-source OpenAI servers par hoti hai, hum sirf data submit karte hain aur ek fine-tuned model-id milta hai.

SECTION 13

RAG vs Fine-tuning — Real Industry Decision

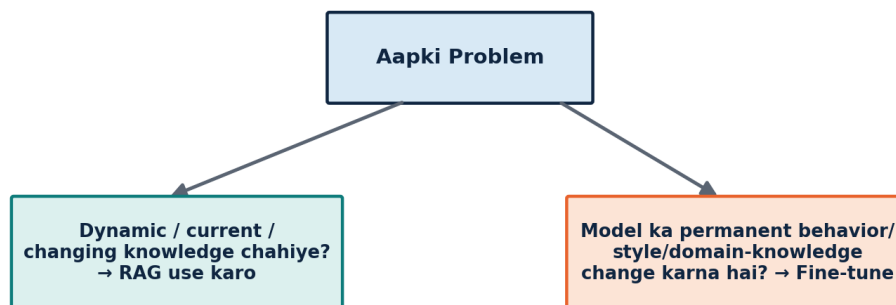
Yeh sabse zyada asked practical interview question hai — “Kab RAG use karoge, kab fine-tuning?”

Industry Reality (Instructor Ke Multiple Classes Se)

- ~90-95% real-world use-cases RAG se solve ho jate hain — koi training overhead nahi
- Fine-tuning tab ki jati hai jab company apna khud ka trained/hosted model chahti ho, third-party APIs par depend nahi karna chahti
- Enterprises zyada tar managed cloud services (Azure OpenAI, AWS Bedrock) use karti hain — data privacy milti hai bina infra overhead ke
- Self-hosted fine-tuned models often underperform karte hain proprietary APIs (GPT/Claude) ke mukable mein
- Sirf ~1-2% companies apne khud ke models/adapters train karti hain, baaki RAG use karte hain

Sahi Decision Kaise Karein?

RAG vs Fine-tuning — Decision Guide



Best Practice: Real projects mein DONO combine karte hain

RAG vs Fine-tuning decision guide

Term	Matlab / Explanation
Fine-tuning chuno agar...	Model ka fundamental knowledge/behavior/style/tone hi change karna ho (domain language, consistent formatting, permanent customization)
RAG chuno agar...	Aapko current/proprietary/dynamic information chahiye jo frequently change hoti ho (documents, policies, live data)
Dono use karo agar...	Company-specific LLM banana ho — fine-tune/instruction-tune model ko company language/style ke liye, aur dynamic documents (SOPs, tickets) ko separate RAG pipeline mein rakho

“False Dichotomy” Samajhna

RAG aur Fine-tuning mutually exclusive nahi hain! Best practice hoti hai combine karna: model ko fine-tune karo company-specific language/workflow/domain terminology ke liye, aur dynamic factual documents ke liye alag se RAG pipeline (vector DB) banao. Upar guardrails/access-control/logging/governance layer bhi add karo.

Deterministic Output Tasks (Important Caveat)

Jahan bohat zyada determinism chahiye ho (jaise SQL/script generation from policy documents), fine-tuning first solution ke tor par recommend nahi ki jati — behtar approach hai: fixed template + RAG + validation/schema-check pipeline + policy classifier for retrieval. Sirf fine-tuning se hallucination/consistency issues fix nahi hoti deterministic output tasks ke liye.

Interview Prep Corner

Interview Sawal	Jawab (Chota aur Clear)
RAG vs Fine-tuning — kab kya use karoge?	RAG jab dynamic/current/frequently-changing knowledge chahiye ho, bina retraining ke. Fine-tuning jab model ka fundamental behavior/style/domain language permanently change karna ho. Real-world mein zyada tar RAG (~90%) use hota hai, aur best results ke liye dono ko combine kiya jata hai.
Kya enterprises apne khud ke models train karte hain?	Zyada tar nahi — sirf ~1-2% companies. Ziyada tar managed APIs (Azure OpenAI, AWS Bedrock, direct OpenAI/Anthropic API) use karte hain data privacy ke sath, bina apna model host kiye.

SECTION 14

Final Interview Cheat-Sheet (Sab Kuch Ek Jagah)

Poori Pipeline Ek Nazar Mein

#	Stage
1	Pretraining — Unsupervised, raw internet-scale data, next-token prediction → Base/Raw Model
2	Instruction Fine-tuning (SFT) — Supervised, instruction/input/output pairs → Instruct Model
3	Preference Alignment — RLHF/DPO, chosen vs rejected → Final Aligned Model
+	Custom Fine-tuning — kisi bhi stage se apna khud ka fine-tuning shuru kar sakte hain

Key Formulas Yaad Rakhein

Concept	Formula/Definition
LoRA	$W = W_0 + BA$ — W_0 frozen, A aur B trainable
Gradient Descent	$W_{\text{new}} = W_{\text{old}} - \eta(\partial L / \partial W)$
Rank	Number of linearly independent rows/columns; max rank = $\min(m, n)$
Quantization	FP32 (4 bytes) → FP16 (2 bytes) → INT8 (1 byte) → INT4 (0.5 bytes)

Framework Comparison

Term	Matlab / Explanation
Hugging Face	Foundation ecosystem — Transformers, Datasets, PEFT, TRL, Accelerate, Bitsandbytes
Unsloth	HF ke upar optimized wrapper — 2-3x faster, 50-80% kam VRAM, exact math (no accuracy loss)
OpenAI API FT	Closed-source, no GPU needed, training OpenAI ke servers par hoti hai

PEFT Methods Quick Recap

Term	Matlab / Explanation
LoRA	Low-rank matrices (A, B) train karta hai, W_0 frozen

Term	Matlab / Explanation
QLoRA	LoRA + 4-bit quantization — kam GPU par bhi chalta hai
DoRA	Weight ko magnitude + direction mein decompose karta hai
Prefix/Prompt Tuning	Learnable vectors/embeddings add karte hain
BitFit	Sirf bias parameters train hote hain

Preference Alignment Quick Recap

Term	Matlab / Explanation
RLHF	PPO-based, RL, 4 models (policy, reference, reward, value), complex
GRPO	DeepSeek ka updated PPO, value model drop, still RL
DPO	Non-RL, sirf 2 models (policy + frozen reference), simple, modern industry choice
ORPO	Non-RL, DPO ka aur advanced/simplified version

30-Second Elevator Answers (Bohat Common Questions)

Interview Prep Corner

Interview Sawal	Jawab (Chota aur Clear)
Fine-tuning kya hoti hai, ek line mein?	Ek already-trained model ko domain/task-specific data par further train karna, taake specific use-case mein behtar perform kare.
LoRA kyun use karte hain, ek line mein?	Poore model ke weights update karne ke bajaye, sirf ek chota low-rank adapter train karte hain — isse memory/cost bohat kam ho jate hain, performance almost same rehti hai.
RAG vs Fine-tuning, ek line mein?	RAG dynamic/current knowledge access ke liye hai, fine-tuning model ka permanent behavior/style/domain-knowledge change karne ke liye. Zyada tar real-world use-cases RAG se hi solve ho jate hain.
Full fine-tuning kyun rarely use hoti hai LLMs ke liye?	Kyunke saare billions parameters update/store karne ke liye itni huge memory/GPU clusters chahiye hote hain ke yeh non-practical aur bohat expensive ho jata hai.
Unsloth vs HF, ek line mein?	Unsloth HF ke upar bana optimized wrapper hai jo custom CUDA/Triton kernels se same accuracy ke sath 2-3x faster training aur kam VRAM deta hai.
DPO RLHF se better kyun mana jata hai aaj kal?	DPO simpler hai (koi reward/value model nahi chahiye, non-RL), training stable aur fast hoti hai, aur results RLHF jaise ya behtar milte hain.

PART 2

RLHF, PPO & DPO — Deep Dive

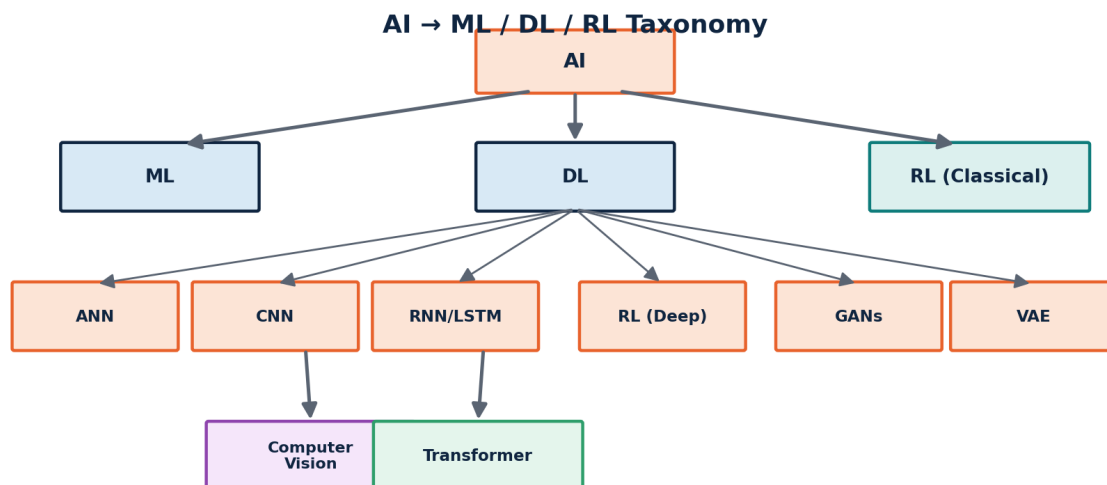
RL Fundamentals se lekar Reward Modeling, PPO ki numeric mechanics, PPO Clipping, OpenAI ka 3-Stage Pipeline, aur DPO ki poori working — sab kuch diagrams ke sath

Class 28 (28 June 2026) ki teen handwritten-notes PDFs se banaya gaya

PART 2 — SECTION 1

RL (Reinforcement Learning) Ke Fundamentals

RLHF samajhne se pehle RL ke basics samajhna zaroori hai. Sab se pehle dekhte hain RL poori AI/ML taxonomy mein kahan fit hoti hai, aur phir iske core concepts.



Transformer → Generative AI → LLMs → Fine-tuning → RAG → Agentic AI

AI se leke Agentic AI tak — RL, DL, Transformers, LLMs sab ka rishta

Yeh diagram dikhata hai ke AI teen branches mein bantee hai: ML, DL, aur RL (Classical). DL age CNN (Computer Vision ke liye), RNN/LSTM (jahan se Transformer nikla), RL(Deep), GANs, aur VAE mein bantee hai. Transformer se hi Generative AI, LLMs, Fine-tuning, RAG, aur Agentic AI tak ka poora safar shuru hota hai — yani jo hum is poore course mein parh rahe hain, woh is taxonomy ke Transformer branch se aata hai.

RL Ki One-Line Definition

“Reinforcement Learning is about learning optimal behavior through rewards and penalties.” Loop: Take action → get feedback → improve decision over time.



Take action → get feedback (reward) → improve decision over time

Agent aur Environment ke beech ka basic RL loop

Is diagram mein Agent (model) Environment ko Action bhejta hai, aur Environment wapas State + Reward deta hai. Yeh cycle baar-baar chalta hai jab tak agent optimal behavior seekh na le.

RL Ke 5 Core Components

Component	Matlab
Agent	The learner (model)
Environment	Where the agent acts (game, chatbot, etc.)
Action	What the agent does
State	Current situation
Reward	Feedback (good or bad)

Poori loop: State → Action → Environment → Reward → Update → Repeat. Real-life analogies: Self-driving car (Agent=car, Action=brake/accelerate/turn, Environment=road traffic, Reward=safe driving); Baby learning to walk (sahi chalne par parents clap karte hain = positive reward, girne par pain = negative reward).

Supervised Learning vs Reinforcement Learning

Concept	Supervised Learning	Reinforcement Learning
Data	SL: Labeled data	RL: No labels
Feedback	SL: Immediate (correct/incorrect)	RL: Delayed reward
Goal	SL: Predict correct output	RL: Maximize reward
Learning Style	SL: Direct learning	RL: Trial & Error
Example	SL: Image classification	RL: Game playing / ChatGPT alignment

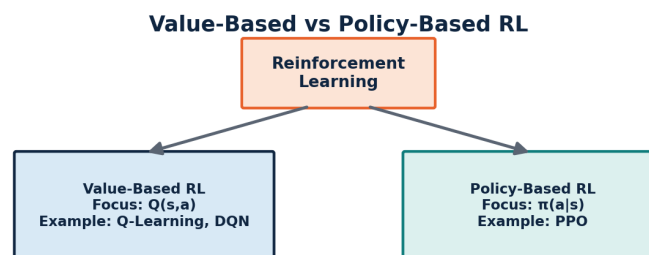
“Supervised learning optimizes against correct answers, while reinforcement learning optimizes based on rewards without knowing the correct answer.” SL = Teacher present (ground truth hoti hai). RL = No teacher, sirf reward system.

RL Evolution Timeline (8 Phases)

Phase	Kya Hua
1950s	Foundations (MDP) — Markov Decision Process, State→Action→Reward→Next State, aaj bhi RL ka backbone
1980s-1990s	Classical RL — Q-Learning, Temporal Difference Learning (tables, scalable nahi)

Phase	Kya Hua
2000s	Slow Progress — mostly theoretical, low compute power, no deep learning support
2013-2015	Deep RL Breakthrough — DQN introduced, neural networks + RL, Atari games solve hue
2016	Policy Gradient & Actor-Critic — value-based se policy-based shift, directly $\pi(a s)$ seekhte hain
2017	PPO (Most Important Phase) — stable training, controlled updates, industry ka core RL algorithm
2018-2020	RL in Real Systems — Robotics, Game AI (AlphaGo, AlphaZero)
2020-2022	RL + LLM (RLHF) — Pipeline: Pretraining→SFT→Reward Model→PPO, ChatGPT-jaisay models train hue

Value-Based vs Policy-Based RL



RL do bade families mein bantee hai: Value-Based aur Policy-Based

Value-Based RL ka focus hota hai $Q(s,a)$ — example: Q-Learning, DQN. Policy-Based RL ka focus hota hai $\pi(a|s)$ — example: PPO. Important: PPO asal mein purely policy-based nahi hai — yeh Actor-Critic hai, yani yeh dono seekhta hai: policy (actor) AND value function (critic).

Q-Learning vs DQN vs PPO

Term	Matlab / Explanation
Q-Learning	Type: Value-Based. Storage: Table. $Q(s,a)$ ek lookup table mein store hota hai. Achi hai small/discrete state spaces ke liye. Problem: state space bara ho to table size explode ho jati hai (scalable nahi).
DQN (Deep Q-Network)	Type: Value-Based, Q-Learning ka upgrade. Key idea: Q-table ko Neural Network se replace karna. Input: State, Output: values for all actions. Large state spaces (images, games) solve karta hai. Limitation: discrete action spaces ke liye behtar, continuous values training unstable ho sakti hai.

Term	Matlab / Explanation
PPO (Proximal Policy Optimization)	Type: Policy-Based (Actor-Critic). Meaning: probability of taking action a given state s . Advantages: continuous actions (robotics) ke sath kaam karta hai, stable training, robotics/games/RLHF (ChatGPT training) mein widely used.

Interview Prep Corner

Interview Sawal	Jawab (Chota aur Clear)
RL ki one-line definition batao?	Reinforcement Learning is about learning optimal behavior through rewards and penalties. Agent action leta hai, environment se feedback (reward) milta hai, aur agent apna decision improve karta jata hai.
SL aur RL mein core farq?	SL mein labeled data aur immediate feedback hota hai (teacher present). RL mein koi labels nahi hote, sirf delayed reward hota hai, aur agent trial-and-error se seekhta hai.
PPO Value-based hai ya Policy-based?	PPO Policy-based hai, lekin purely policy-based nahi — yeh Actor-Critic hai, jo policy (actor) aur value function (critic) dono seekhta hai.

PART 2 — SECTION 2

Sirf SFT Ki Problem aur Reward Modeling

Sirf SFT Se Kya Problem Hai?

SFT task performance improve karta hai, lekin yeh guarantee nahi karta ke model ka behavior human values/safety/expectations ke sath align ho. Pretraining model ko “knowledgeable” banati hai, lekin “controllable ya trustworthy” nahi.

SFT ki Limitation — France/Germany Example (Bohat Important)

SFT training data: User: “What is the capital of France?” → Model taught: “Paris is the capital of France.” — Bas itna hi model dekhta hai, sirf correct answer. Ab real world mein: User: “What is the capital of Germany?” — Model do outputs de sakta hai: “Paris is the capital of Germany” (galat) ya “Berlin is the capital of Germany” (sahi). SFT sirf yeh sikhata hai “Paris is correct for France” — lekin yeh explicitly NAHI sikhata “Paris is wrong for Germany.” Isi wajah se during SFT, achi answers ki probability barhti hai, lekin buri answers ki probability bhi accidentally barh sakti hai.

Preference alignment ensure karta hai model: Helpful, Harmless, Honest, Polite, Safe, Non-toxic, aur Consistent with human expectations ho. “Preference alignment bridges the gap between 'knows language' and 'behaves well'.”

RLHF vs SFT — Core Distinction

SFT teach karta hai: “This is the correct answer.” RLHF teach karta hai: “This answer is better than the other one.” Real world mein multiple correct answers ho sakte hain.

Worked Example — “Explain AI”

Output A (simple, clear): “AI means machines that can learn, think, or make decisions like humans...” vs Output B (complex, confusing): “AI is a computational paradigm involving algorithmic approximations...”

Dono technically correct hain, lekin Human Output A ko prefer karta hai. Yehi RLHF ka kaam hai — correctness nahi, preference/clarity/style ko align karna, jo SFT akela capture nahi kar sakta.

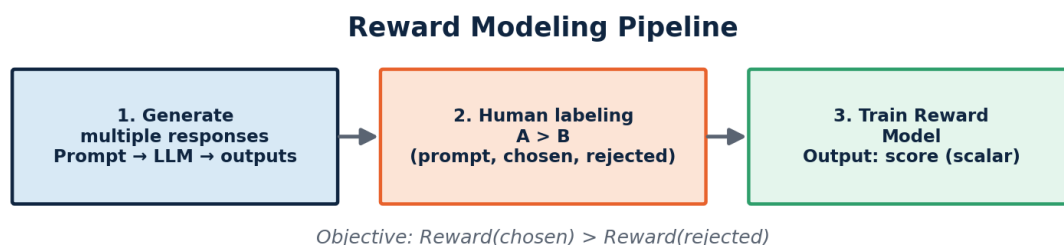
Reward Modeling — Step 1: Human Preferences Collect Karna

Human ko ek prompt diya jata hai: “Explain Machine Learning.” LLM do responses generate karta hai — Response A aur Response B. Human evaluator kehta hai “A is better than B.” Hum store karte hain: Prompt, Chosen=A, Rejected=B — isay Human Feedback (HF) kehte hain, aur isi se ek preference dataset banta hai (reference: huggingface.co/datasets/Anthropic/hh-rlhf).

Super Important Interview Question

Q: “Does PPO directly use Human Feedback?” — A: No. Human Feedback PPO ke andar directly use NAHI hoti. Human Feedback pehle Reward Model ko train karti hai. Phir PPO Reward Model ke scores ko reward ki tarah use karta hai. “The 'HF' in RLHF is used to train the Reward Model. PPO never interacts with humans directly.”

Reward Modeling Pipeline



Reward Model banane ke 3 steps

Step 1: Prompt → LLM → multiple responses generate hoti hain. Step 2 (VERY IMPORTANT): Human compare karta hai $A > B$ → dataset banta hai (prompt, chosen, rejected). Step 3: Reward Model train hota hai — Input: (prompt+response), Output: score (scalar value). Objective: $\text{Reward}(\text{chosen}) > \text{Reward}(\text{rejected})$.

Core Idea: correct vs incorrect ki jagah, hum better vs worse seekhte hain. “An LLM does not naturally know what is 'better'.” Poori chain: Human → Preference Data → Reward Model → Reward Score → PPO.

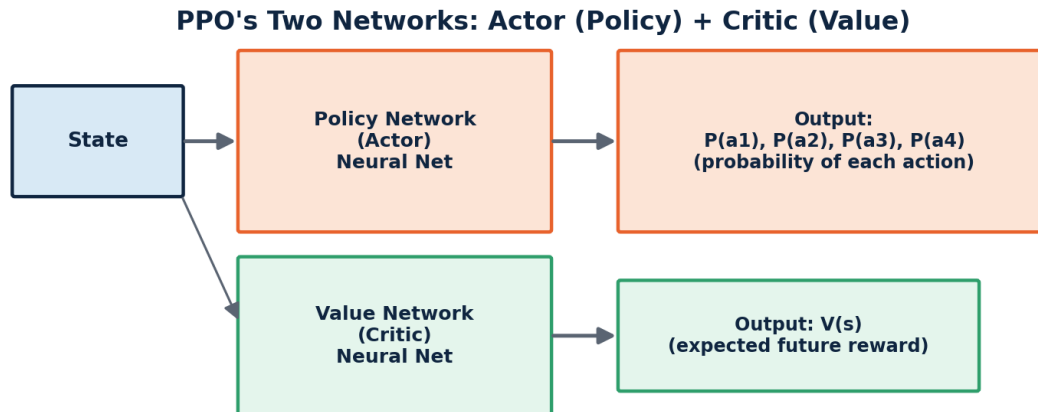
What is Reward Modeling? Ek model ko train karna jo responses ko human preferences ke mutabiq evaluate/score/rank kar sake. In RLHF: Reward Model = a proxy for human judgment.

Interview Prep Corner

Interview Sawal	Jawab (Chota aur Clear)
Sirf SFT se kya problem hai?	SFT sirf 'correct answer' sikhata hai, lekin explicitly 'yeh galat hai' nahi sikhata. Isliye model ko yeh nahi pata hota ke do sahi jawabon mein se konsa behtar hai, ya konsa response human values ke khilaf hai.
Reward Model kya hai?	Ek trained model jo (prompt, response) dekh kar ek scalar score deta hai jo batata hai response kitna 'acha' hai. Yeh human judgment ka proxy hai, human preferences se seekha jata hai.

PPO Ki Poori Mechanics — Numeric Walkthrough

PPO Ke Do Networks



Policy Network (Actor) aur Value Network (Critic) — PPO ke do neural networks

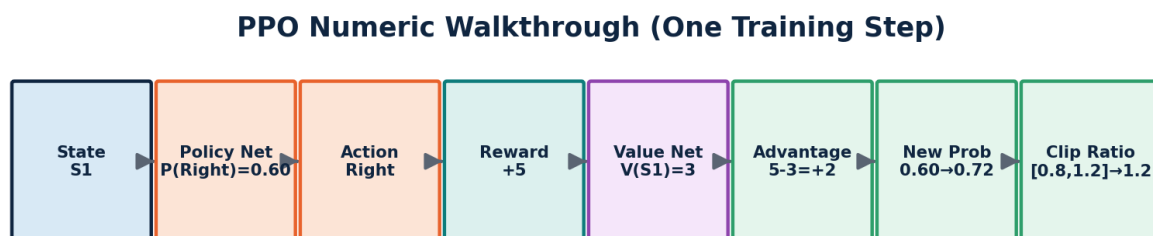
Policy Network (Actor): Input=State, Output=har action ki probability ($P(a_1)$, $P(a_2)$...).

Value Network (Critic): Input=State, Output= $V(s)$ — “is state se kitna future reward mil sakta hai” ka estimate.

Proximal ka matlab — sab se important idea: policy ko ek dafa mein bohat zyada mat badlo. Bad approach: 70%→100% (bohat aggressive). Good PPO approach: 70%→75% (chota, safe update). Isi se training stable rehti hai.

PPO Numeric Walkthrough — Ek Poora Worked Example

Grid-world robot example se PPO ki har step numerically samajhte hain (yeh instructor ka canonical teaching example hai, 3 baar alag context mein use hua):



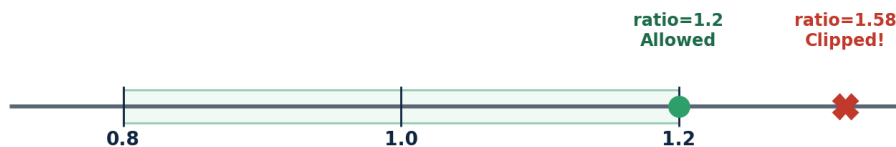
$$\text{Ratio} = \text{new_prob} / \text{old_prob} = 0.72 / 0.60 = 1.2 \rightarrow \text{within } [0.8, 1.2] \text{ allowed range, so update is accepted}$$

Grid-world robot example — instructor ka canonical PPO teaching example

Term	Matlab / Explanation
1. Policy Network decides	State=S1. Policy output: P(Left)=0.10, P(Right)=0.60, P(Up)=0.20, P(Down)=0.10 → Right highest → Action=Right, old_prob=0.60 store
2. Environment gives reward	Robot moves Right → Reward=+5, Next State=S2. Stored: (S1, Right, +5, old_prob=0.60)
3. Value Network predicts	Value Network ka kaam: "is state se kitna future reward mil sakta hai?" → V(S1)=3 (expected)
4. Advantage calculate	Advantage = Actual Return – Expected Value = 5 – 3 = +2 — matlab Right expected se behtar tha
5. Policy update	Old P(Right)=0.60 → New P(Right)=0.72. Ratio = new/old = 0.72/0.60 = 1.2 — Right ab 20% zyada likely hai
6. PPO Clipping check	Allowed range = [0.8, 1.2]. Hamara ratio=1.2 → Allowed. Agar P(Right)=0.90 hota to ratio=1.5 → Too Large → Clip hoga

PPO Clipping — Visual aur Asal Mechanics

PPO Clipping: Allowed Ratio Range $[1-\epsilon, 1+\epsilon]$ ($\epsilon=0.2$)



Allowed ratio range [0.8, 1.2] — is se bahar jaane par PPO clip kar deta hai

PPO Clipping Ki Asal Mechanics (Important Nuance)

PPO clipping seedha neural network weights ko rokta nahi hai. Balke PPO apni loss function aisay design karta hai ke jab ratio > 1.2 ho jaye, model ko probability aur barhane ka koi extra benefit nahi milta — is se gradient chota ho jata hai, weight update chota hota hai, aur naturally new probability zyada jump nahi karti.

Real intuition: Bina clipping ke: 0.60 → 0.95 (huge jump, unstable). PPO clipping ke sath: 0.60 → 0.68 → 0.72 → 0.75 (gradual learning, stable) — multiple steps mein slowly improve hota hai.

PPO Supervised Learning nahi hai kyunke yeh ground-truth labels se nahi seekhta — yeh environment ya reward model se milne wale reward signals se seekhta hai.

PPO Overall Training Loop

- 1. Compute loss for Value Function Network
- 2. Compute loss for Policy Network

- 3. Update both networks
- 4. Repeat

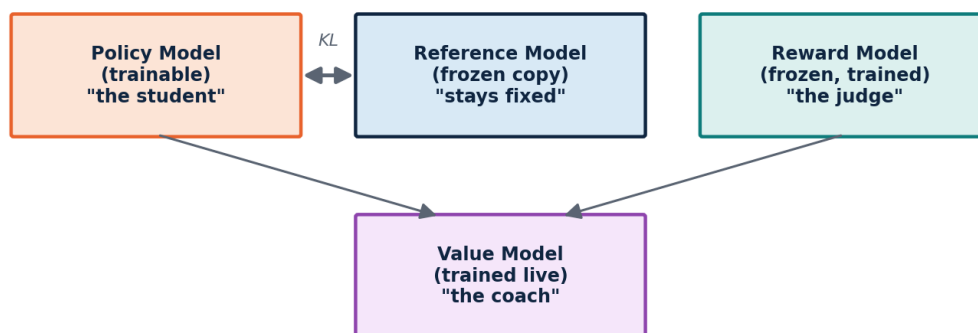
Interview Prep Corner

Interview Sawal	Jawab (Chota aur Clear)
Proximal Policy Optimization mein 'Proximal' ka kya matlab hai?	Matlab hai policy ko ek hi step mein bohat zyada mat badlo — sirf chote, safe updates karo (jaise 70%→75%, na ke 70%→100%). Isse training stable rehti hai.
Advantage kaise calculate hota hai?	Advantage = Actual Reward – Expected Value (jo Value Network predict karta hai). Positive advantage ka matlab action expected se behtar tha.
PPO clipping asal mein kya karta hai?	Yeh directly weights ko block nahi karta, balke loss function ko is tarah design karta hai ke ratio ek range (jaise 0.8-1.2) se bahar jaane par gradient chota ho jata hai, jisse update bhi chota hota hai.

RLHF Ka Poora Pipeline — 4 Models aur Cast of Characters

PPO Stage Mein 4 Models Use Hote Hain

The 4 Models Used in PPO Training



Policy generates a response; Reward Model scores it; Value Model predicts expected reward; Reference Model keeps the policy from drifting too far (KL Divergence)

Policy, Reference, Reward, aur Value — PPO ke 4 models

Model	Kya Hai
Policy Model (Actor)	Woh actual LLM jo train ho raha hai — "Full LLM"
Reference Model	Policy model ki frozen copy — sirf KL divergence measure karne ke liye, khud train nahi hota
Reward Model	Already trained, frozen — "Smaller LLM + Reward Head", actual reward deta hai
Value Model (Critic)	PPO ke doran train hota hai — "Policy Model + Value Head", expected reward predict karta hai

KL Divergence/Penalty measure karta hai ke policy model apne original (reference) behavior se kitna "drift" kar gaya — taake model random/unrelated responses dena shuru na kar de.

Complete RLHF Pipeline (One-Liner)

Humans → Preference Data → Reward Model Training → Reward Model → PPO Training → Policy Update → Better LLM

OpenAI InstructGPT / RLHF 3-Stage Pipeline

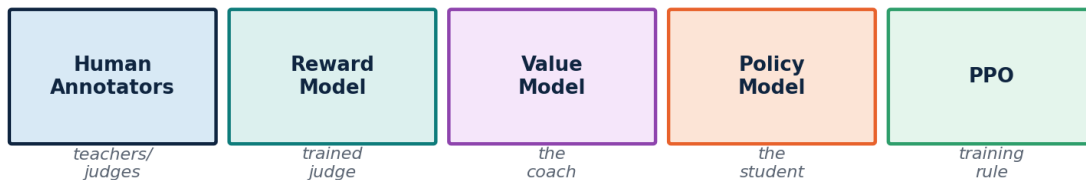


OpenAI InstructGPT ka classic 3-stage RLHF pipeline

Stage 1 (SFT): Human demonstration data se base LLM ko supervised fine-tune karte hain. Stage 2 (Reward Model): Model multiple outputs generate karta hai, human unhe rank karta hai, is se Reward Model train hota hai. Stage 3 (PPO/RL): Policy Model naye prompts par response generate karta hai, Reward Model use score karta hai, aur PPO us reward ko use kar ke Policy ko update karta hai.

Cast of Characters — RLHF Ke 5 Kirdar (Yaad Rakhne Ka Best Tareeqa)

RLHF Cast of Characters



RLHF ke 5 main characters, ek analogy ke sath

Character	Role
Human Annotators	The teachers/judges
Reward Model	The trained judge
Value Model	The coach jo predict karta hai student kitna achha karega
Policy Model	The student
PPO	The training rule jo student ko safely update karta hai

Interview Prep Corner

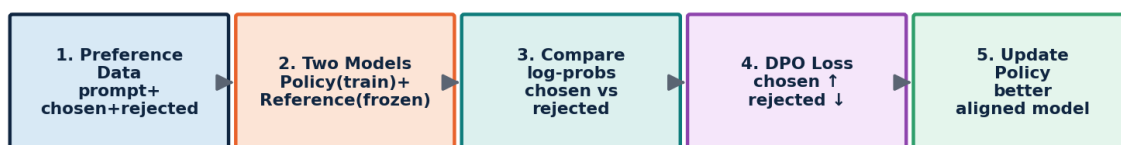
Interview Sawal	Jawab (Chota aur Clear)
PPO stage mein kitne models use hote hain aur kaunse?	4 models: Policy Model (trainable, actual LLM), Reference Model (frozen copy, KL divergence ke liye), Reward Model (frozen, trained judge), aur Value Model (live train hota hai, expected reward predict karta hai).
KL Divergence kyun zaroori hai?	Yeh policy model ko reference model se bohat zyada drift hone se rokta hai — taake model apni original coherent behavior na khoye aur random/unrelated responses dena shuru na kare.
OpenAI ka RLHF pipeline 3 stages mein batao?	Stage 1: SFT (human demonstrations se base model banate hain). Stage 2: Reward Model training (human preference rankings se). Stage 3: PPO/RL (reward model ke scores se policy ko update karte hain).

PART 2 — SECTION 5

DPO — Direct Preference Optimization (Deep Dive)

Simple idea: model ko sikhana ke chosen answer ko rejected answer se zyada prefer kare. DPO RLHF ka modern, simple alternative hai — koi reward model, koi PPO loop nahi chahiye.

DPO (Direct Preference Optimization) — 5 Steps



Simple idea: make the model prefer the chosen answer over the rejected answer

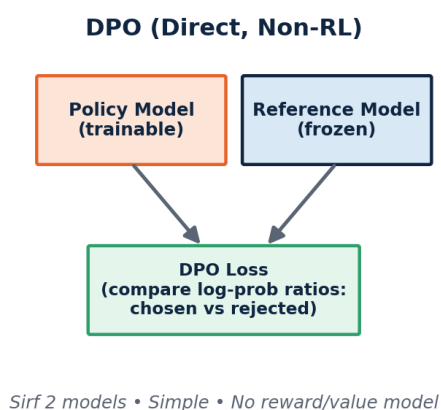
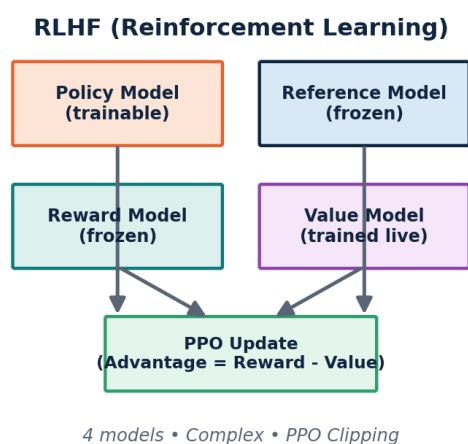
DPO ke 5 steps — data se le kar policy update tak

- 1. Preference Data: Prompt, Chosen Answer, Rejected Answer
- 2. Two Models: Policy Model (Trainable, yeh update hota hai) aur Reference Model (Frozen SFT Model, fixed rehta hai — ya koi bhi frozen version/different model bhi ho sakta hai)
- 3. Compare Preferences: dono models log-probabilities compute karte hain: $\log P(\text{chosen})$ aur $\log P(\text{rejected})$. DPO check karta hai ke policy chosen ko rejected se kitna zyada prefer karta hai reference ke mukable mein — Implicit reward $\approx \beta \times \log(\text{policy/reference})$
- 4. DPO Loss: Loss kam hoti hai jab chosen > rejected (policy mein). Effect: chosen ki probability barhti hai, rejected ki ghatti hai. β (beta) control karta hai model reference se kitna door move kare
- 5. Update Policy Model: Updated model — chosen-style answers zyada generate karega, rejected-style kam, aur human preferences ke sath behtar aligned hoga

DPO vs RLHF (Yaad Rakhein)

No separate reward model • No PPO loop • Simpler aur more stable training.

“SFT teaches how to answer. DPO teaches which answer is better.”



RLHF (4 models, RL-based) vs DPO (2 models, Non-RL, simple direct loss)

TRL Library mein Implementation

TRL Class	Kis liye
RewardTrainer / RewardConfig	Reward model train karne ke liye (RLHF Stage 2)
PPOTrainer / PPOConfig	PPO training ke liye (4 models pass karte hain)
DPOTrainer / DPOConfig	DPO training ke liye (koi external reward model nahi chahiye)

Part 2 Ka Final Interview Cheat-Sheet

Sab Se Zyada Poochay Jaane Wale Sawal

Interview Prep Corner

Interview Sawal	Jawab (Chota aur Clear)
Does PPO directly use Human Feedback?	No. Human Feedback sirf Reward Model train karne ke liye use hoti hai. PPO sirf Reward Model ke scores ko reward ki tarah use karta hai — kabhi humans se directly interact nahi karta.
RLHF aur DPO mein basic farq kya hai?	RLHF Reinforcement Learning based hai, 4 models (policy, reference, reward, value) chahiye, PPO algorithm use karta hai. DPO non-RL, simple supervised technique hai, sirf 2 models (policy + frozen reference) chahiye, direct loss function se train hota hai.
SFT aur RLHF mein core farq kya hai?	SFT teach karta hai 'yeh correct answer hai.' RLHF teach karta hai 'yeh answer doosre se behtar hai.' Real world mein multiple correct answers ho sakte hain, jinhe SFT akela differentiate nahi kar sakta.
PPO clipping kyun zaroori hai?	Taake policy ke weights mein ek hi step mein bohat bada/destructive update na ho jaye — yeh training ko stable rakhta hai.
Advantage kya hota hai PPO mein?	Advantage = Actual Reward (Reward Model se) minus Expected Reward (Value Model se). Positive advantage matlab action expected se behtar tha, isliye uski probability barhai jati hai.
Aaj kal industry mein kaunsi technique zyada use hoti hai?	DPO aur ORPO — kyunke woh RLHF/PPO se mechanically bohat simpler hain aur similar ya behtar results dete hain.